

Obs: o nome real seria (Minhas Tarefas — Sistema de Gerenciamento de Tarefas “O objetivo continua exatamente de acordo com o solicitado pelo professor: desenvolver um sistema em que usuários autenticados possam criar e gerenciar suas próprias tarefas.”)

TASKFLOW — SISTEMA DE GERENCIAMENTO DE TAREFAS

1. Descrição do Sistema

O **TaskFlow** é um sistema web desenvolvido para auxiliar usuários na organização e no gerenciamento de suas tarefas pessoais.

O sistema permite que cada usuário realize seu cadastro e login utilizando e-mail e senha. Após a autenticação, o usuário poderá cadastrar, visualizar, atualizar e excluir suas próprias tarefas.

Cada tarefa possuirá informações como título, descrição, data limite, prioridade e status. Além disso, o sistema será desenvolvido de forma que cada usuário tenha acesso somente às tarefas associadas à sua conta.

O projeto será estruturado utilizando **React com Vite no frontend, Python com FastAPI no backend e Supabase para banco de dados PostgreSQL e autenticação dos usuários.**

2. Objetivo Geral

Desenvolver um sistema web distribuído para gerenciamento de tarefas, permitindo que usuários autenticados possam organizar e acompanhar suas atividades de maneira simples e individual, utilizando uma arquitetura com frontend, backend e banco de dados separados.

3. Objetivos Específicos

- Permitir o cadastro de usuários por meio de e-mail e senha;
- Permitir que usuários cadastrados realizem login no sistema;
- Permitir o cadastro de novas tarefas;
- Permitir a visualização das tarefas cadastradas;
- Permitir a edição das informações de uma tarefa;
- Permitir a exclusão de tarefas;
- Classificar as tarefas de acordo com sua prioridade;
- Permitir o acompanhamento do status de cada tarefa;

- Garantir que cada usuário tenha acesso somente às suas próprias tarefas;
- Apresentar mensagens de sucesso ou erro durante as operações;
- Implementar testes automatizados para verificar as principais funcionalidades do backend.

4. Público-Alvo

O TaskFlow poderá ser utilizado por qualquer pessoa que necessite organizar suas atividades e compromissos, como estudantes, professores e profissionais.

5. Funcionalidades

5.1 Cadastro

O usuário poderá criar uma conta utilizando:

- E-mail;
- Senha.

5.2 Login

O usuário cadastrado poderá acessar o sistema informando seu e-mail e senha.

A autenticação será realizada utilizando o Supabase.

5.3 Gerenciamento de Tarefas

Depois de autenticado, o usuário poderá realizar as quatro operações básicas de um CRUD:

Create — Criar: cadastrar uma nova tarefa.

Read — Ler: visualizar suas tarefas cadastradas.

Update — Atualizar: alterar uma tarefa existente.

Delete — Excluir: remover uma tarefa.

6. Informações das Tarefas

Cada tarefa possuirá:

- Título;
- Descrição;
- Data limite;
- Prioridade;
- Status.

Prioridades

As prioridades disponíveis serão:

- Baixa;
- Média;
- Alta.

Status

Os possíveis status serão:

- Pendente;
- Em andamento;
- Concluída.

7. Tecnologias Utilizadas

Frontend

React + Vite

Responsável pela interface visual e pela interação do usuário com o sistema.

Backend

Python + FastAPI

Responsável pela API REST, regras da aplicação e operações de gerenciamento das tarefas.

Banco de Dados

Supabase/PostgreSQL

Responsável pelo armazenamento das tarefas.

Autenticação

Supabase Auth

Responsável pelo cadastro, login e autenticação dos usuários utilizando e-mail e senha.

Testes

Pytest

Responsável pela execução dos testes automatizados do backend.

8. Arquitetura do Sistema

O TaskFlow será dividido em três partes principais:

Usuário → Frontend React → API FastAPI → Supabase/PostgreSQL

O usuário interage com a interface desenvolvida em React. O frontend envia requisições HTTP para a API REST desenvolvida com FastAPI. A API processa as operações necessárias e utiliza o Supabase para armazenamento dos dados.

9. Fluxo Básico do Sistema

1. O usuário acessa o TaskFlow;
2. Realiza seu cadastro ou login;
3. O Supabase autentica o usuário;
4. O usuário acessa a página de tarefas;
5. O usuário pode cadastrar uma nova tarefa;
6. O frontend envia a solicitação para o backend;
7. O FastAPI processa a requisição;
8. Os dados são armazenados no PostgreSQL do Supabase;
9. O backend retorna o resultado;
10. O frontend atualiza a interface;
11. Uma mensagem de sucesso ou erro é apresentada ao usuário.

10. Estrutura do Projeto

O projeto será organizado em duas pastas principais:

```
taskflow/  
├── client/  
│   └── Frontend React + Vite  
├── server/  
│   └── Backend Python + FastAPI  
└── README.md
```

11. Testes do Sistema

O backend possuirá testes automatizados utilizando Pytest.

Serão realizados, no mínimo, testes de:

- Autenticação e autorização;
- Criação de tarefas;
- Listagem de tarefas;
- Atualização de tarefas;
- Exclusão de tarefas.

Também será verificado se um usuário não consegue acessar ou modificar tarefas pertencentes a outro usuário.

12. Resultado Esperado

Ao final do desenvolvimento, espera-se obter um sistema web funcional no qual diferentes usuários possam criar suas contas e gerenciar suas próprias tarefas de maneira independente.

O sistema deverá demonstrar a comunicação entre frontend, backend e banco de dados, além da utilização de autenticação e testes automatizados.

COMANDOS USADOS:

crie a pasta do backend

No mesmo terminal:

```
mkdir server
```

Entre nela:

```
cd server
```

Agora sua estrutura ficará:

```
minhas-tarefas/  
└─ server/
```

O professor pede justamente que o projeto final possua `client/` para React e `server/` para FastAPI.

4. Criar o ambiente virtual do Python

Dentro de `server`, execute:

```
python -m venv venv
```

No Windows, depois execute:

```
venv\Scripts\activate
```

Se der certo, provavelmente aparecerá algo como:

```
(venv)
```

antes do caminho no terminal.

Exemplo:

```
(venv) C:\Users\...\minhas-tarefas\server>
```

5. Instalar o FastAPI

Agora execute:

```
pip install "fastapi[standard]"
```

Essa é atualmente uma das formas recomendadas pela própria documentação do FastAPI.

 **Supabase**
<https://supabase.com> · [Traduzir esta página](#)

Supabase | The Postgres Development Platform

Build production-grade applications with a Postgres database, Authentication, instant APIs, Realtime, Functions, Storage and Vector embeddings.

Pricing >
Explore Supabase fees and pricing information. Find our ...

Docs >
Supabase is the Postgres development platform providing ...

Beginners >
Supabase gives you the tools to easily manage databases ...

Database >
Every Supabase project is a full Postgres database with realtime ...

Data REST API >
Supabase auto-generates an API directly from your database ...

Mais resultados de supabase.com »

Clica em “Comece seu projeto”

Depois

CLICA EM “Continuar com o GitHub”

Vai colocar e conectar a seu GitHub

Depois

Criar uma nova organização
As organizações são uma forma de agrupar seus projetos. Cada organização pode ser configurada com diferentes membros de equipe e configurações de faturamento.

Nome

Qual o nome da sua empresa ou equipe? Você pode alterá-lo mais tarde.

Tipo

Pessoal

Qual das opções a seguir melhor descreve a sua organização?

Plano

Grátis - \$0/mês

Qual plano melhor se adapta às necessidades da sua organização?
[Saiba mais](#).

Cancelar

Criar organização

Preencha exatamente assim:

- **Nome:** Projetos Faculdade
- **Tipo:** deixe **Pessoal**
- **Plano:** deixe **Grátis – \$0/mês**

Próximo

Senha do banco de dados

Digite uma senha forte.

Essa é a senha do seu banco de dados Postgres, portanto, ela deve ser forte e difícil de adivinhar. [Gerar uma senha.](#)

Região

Américas

Selecione a região mais próxima dos seus usuários para obter o melhor desempenho.

Segurança

- ☒ **Habilitar API de Dados**
Gere automaticamente uma API RESTful para seu esquema público. Recomendado se estiver usando uma biblioteca cliente como o `supabase-js`.
- ☒ **Expor automaticamente novas tabelas**
Concede privilégios às funções da API de Dados por padrão, expondo novas tabelas. Recomendamos desativar essa opção para controlar o acesso manualmente.
- ☐ **Ativar RLS automático**
Crie um gatilho de evento que habilite automaticamente a Segurança em Nível de Linha em todas as novas tabelas no esquema público.

CONFIGURAÇÃO AVANÇADA >

Cancelar Criar novo projeto

Criar um novo projeto

Seu projeto terá sua própria instância dedicada e um banco de dados Postgres completo. Uma API será configurada para que você possa interagir facilmente com seu novo banco de dados.

Organização

Projetos Faculdade LIVRE

GitHub (opcional)

Conecte-se ao GitHub

Ideal para fluxos de trabalho com foco no agente: atualize seu esquema no código, envie-o para o GitHub e o Supabase implantará as alterações automaticamente. [Saiba mais](#)

1. Senha do banco de dados: precisa criar uma senha forte. Você pode clicar em “Gerar uma senha”. Depois **copie e guarde essa senha em um lugar seguro**, pois poderemos precisar dela mais adiante. Não precisa me enviar a senha.

2. Região: pode deixar “Américas” como está.

3. Segurança: na sua tela aparecem três opções. Para o nosso projeto, deixe:

- ☒ **Habilitar API de Dados**
- ☒ **Expor automaticamente novas tabelas** — desmarque, pois a própria tela recomenda desativar para controlar o acesso manualmente.
- ☒ **Ativar RLS automático** — marque. O RLS (*Row Level Security*) será importante porque nosso sistema precisa garantir que cada usuário veja apenas suas próprias tarefas, exatamente como o trabalho exige.
- **Organização:** deixe `Projetos Faculdade` como está. ☒

- **GitHub (opcional): não precisa conectar agora.** Pode deixar sem mexer.
Vamos configurar o GitHub depois, quando o sistema já estiver funcionando.

Então:

Organização: Projetos Faculdade ☒

GitHub: deixar sem conectar ☒